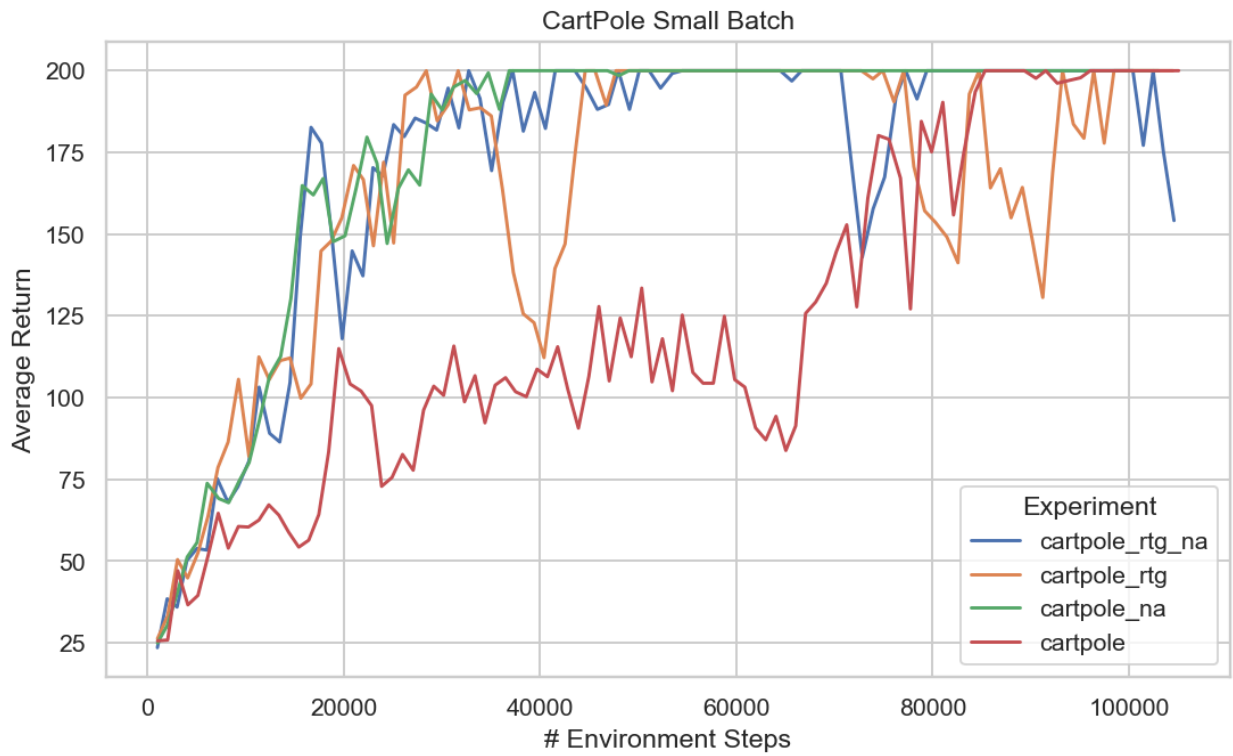
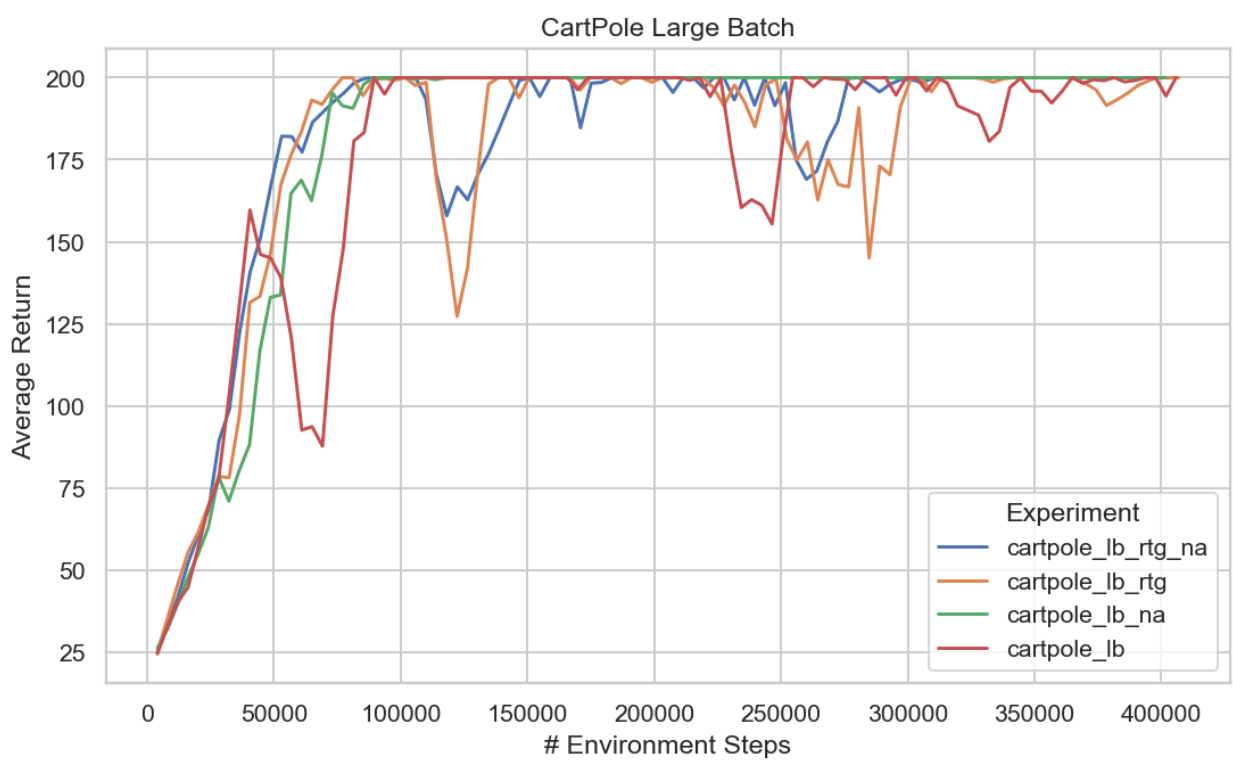
CS285 HW2 Report

Pranav Nutalapati

February 27, 2026

3 Vanilla Policy Gradients





Questions

- **Which value estimator has better performance without advantage normalization: the trajectory-centric one, or the one using reward-to-go?**

  Reward-to-go performed better than trajectory centric for both batch sizes.

- **Between the two value estimators, why do you think one is generally preferred over the other?**

  Reward-to-go is preferred because it disregards reward from past actions, only focusing future rewards. This is generally more useful since only want to consider rewards that can be affected by the current action.

- **Did advantage normalization help?**

  Not uniformly. It had a large impact for trajectory-centric PG in small batch, and modestly in large batch. With RTG, it's hard to tell because the differences are minor and possibly just run-to-run variance.

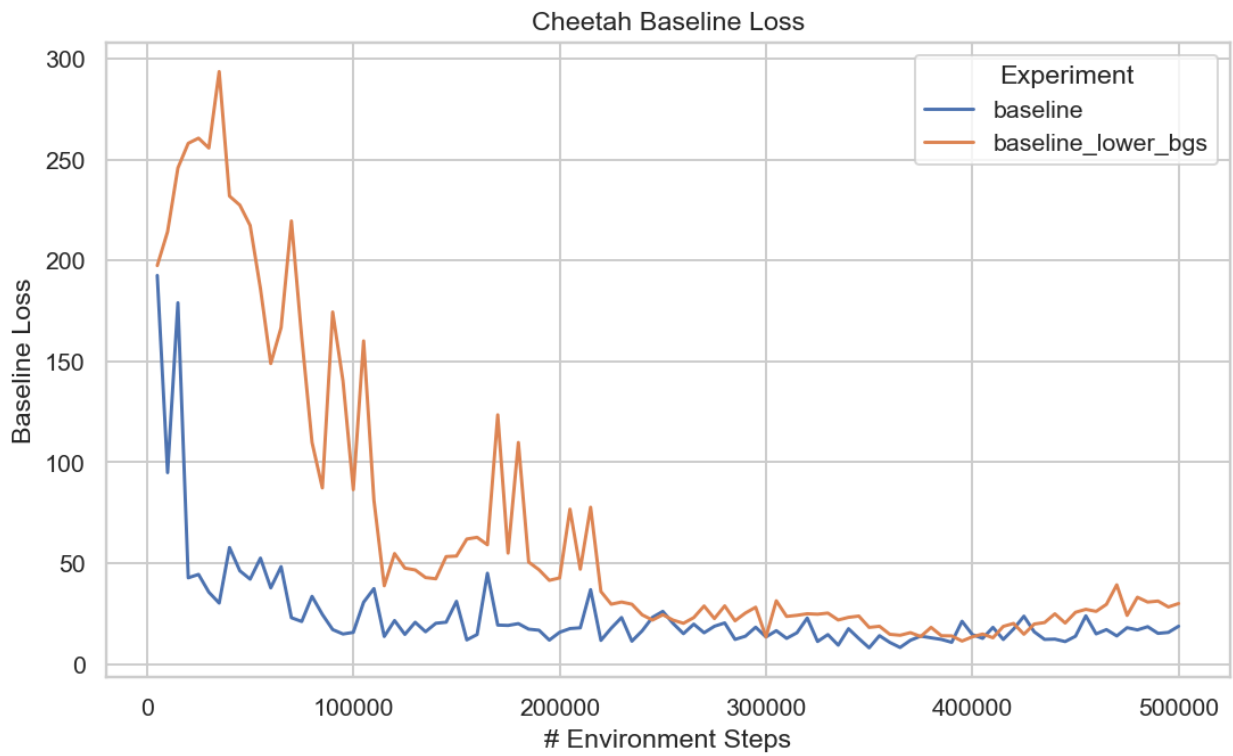
- **Did the batch size make an impact?**

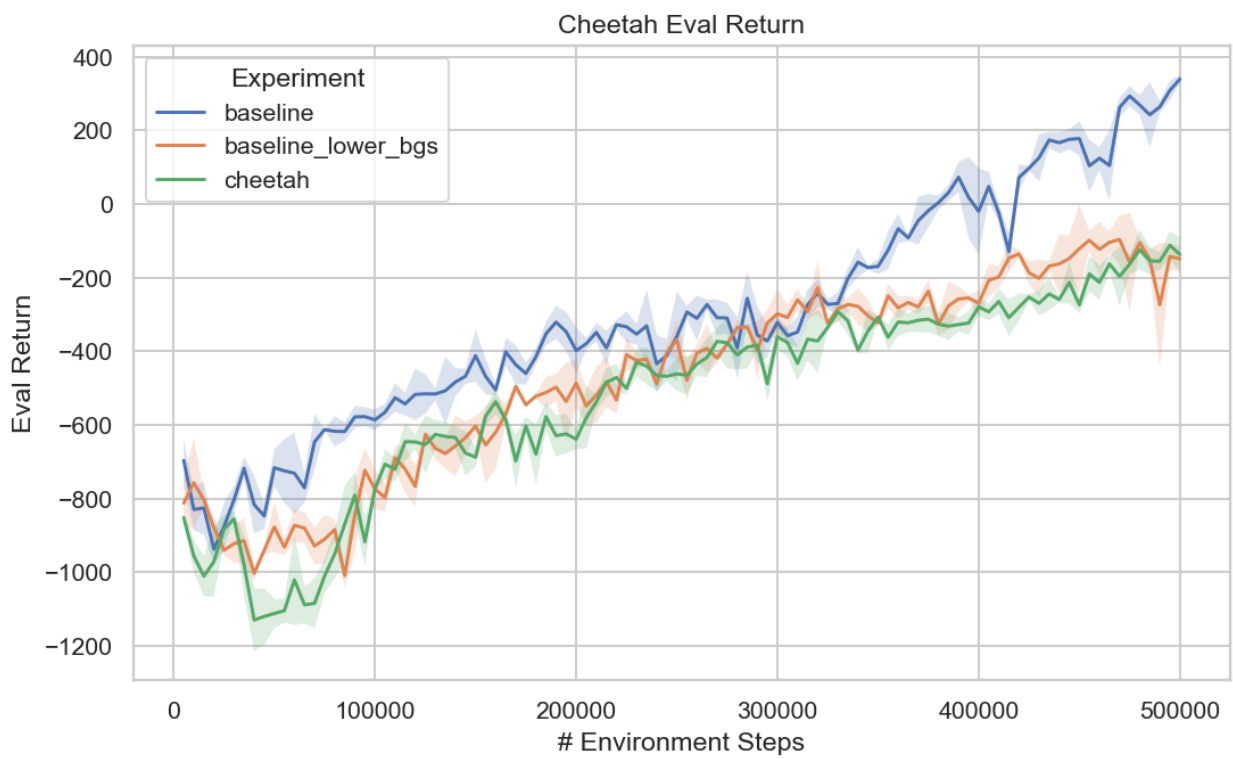
  Although all configurations eventually converged, larger batch sizes needed more environment steps than smaller batch sizes.

Configuration

```
uv run src/scripts/run.py --env_name CartPole-v0 --exp_name cartpole \
  -n 100 -b 1000

uv run src/scripts/run.py --env_name CartPole-v0 --exp_name cartpole_rtg \
  -n 100 -b 1000 -rtg

uv run src/scripts/run.py --env_name CartPole-v0 --exp_name cartpole_na \
  -n 100 -b 1000 -na

uv run src/scripts/run.py --env_name CartPole-v0 --exp_name cartpole_rtg_na \
  -n 100 -b 1000 -rtg -na

uv run src/scripts/run.py --env_name CartPole-v0 --exp_name cartpole_lb \
  -n 100 -b 4000

uv run src/scripts/run.py --env_name CartPole-v0 --exp_name cartpole_lb_rtg \
  -n 100 -b 4000 -rtg

uv run src/scripts/run.py --env_name CartPole-v0 --exp_name cartpole_lb_na \
  -n 100 -b 4000 -na

uv run src/scripts/run.py --env_name CartPole-v0 --exp_name cartpole_lb_rtg_na \
  -n 100 -b 4000 -rtg -na
```

4 Neural Network Baseline





Questions

- **How does a decreased number of baseline gradient steps affect the baseline learning curve and the performance of the policy?**

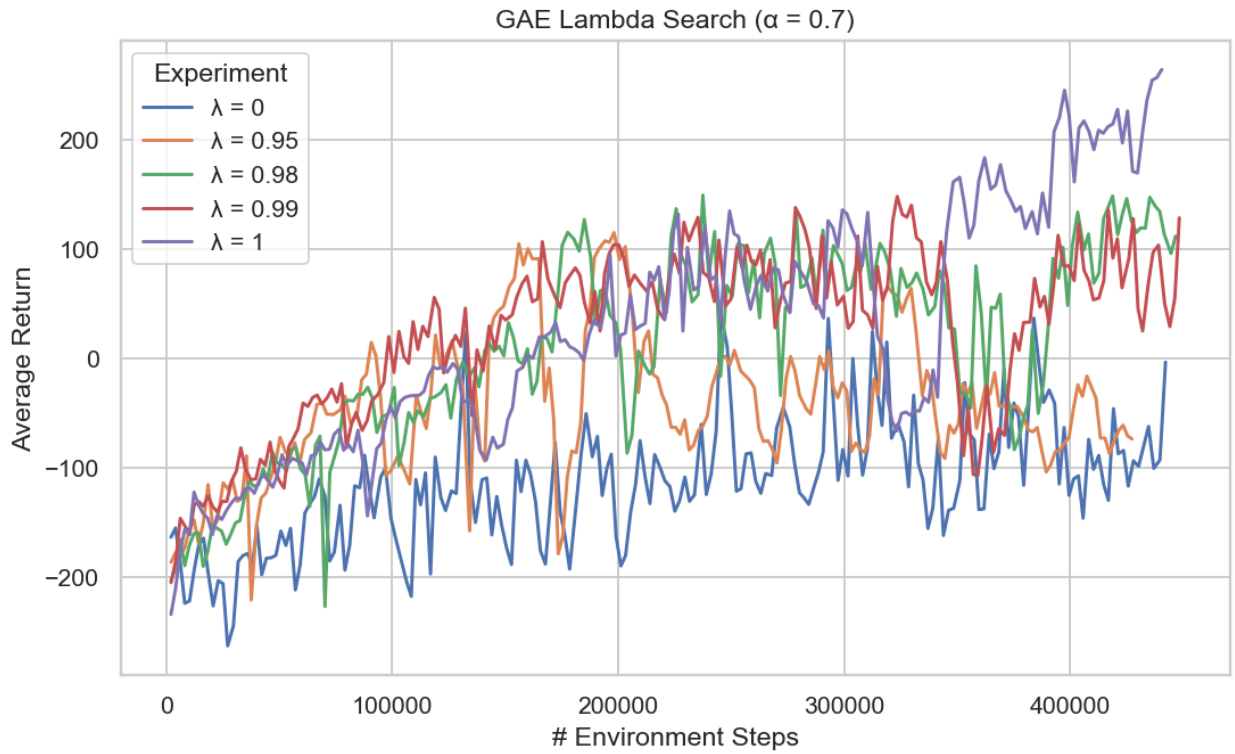
  Lowering the baseline gradient steps from 5 to 1 results in a clearly worse-performing model. The baseline loss was higher earlier and overall when using the lower BGS value, and the policy never reached a non-negative eval return.

  The best eval return for `-bgs 1` was -96.5, while `-bgs 5` reached 339.3 in the same number of environment steps.

Configuration

```
# No baseline
uv run src/scripts/run.py --env_name HalfCheetah-v4 --exp_name cheetah \
  -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01

# Baseline
uv run src/scripts/run.py --env_name HalfCheetah-v4 --exp_name cheetah_baseline \
  -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 5

# Fewer baseline gradient steps (5 -> 1)
uv run src/scripts/run.py --env_name HalfCheetah-v4 --exp_name cheetah_baseline_lower_bgs \
  -n 100 -b 5000 -eb 3000 -rtg --discount 0.95 -lr 0.01 --use_baseline -blr 0.01 -bgs 1
```

5 Generalized Advantage Estimation



Questions

- **How did λ affect performance?**

  Higher λ generally improved performance and stability in my LunarLander runs. Low λ values underperformed, while $\lambda = 1$ achieved the best peak and final returns.

- **What do $\lambda = 0$ and $\lambda = 1$ correspond to?**

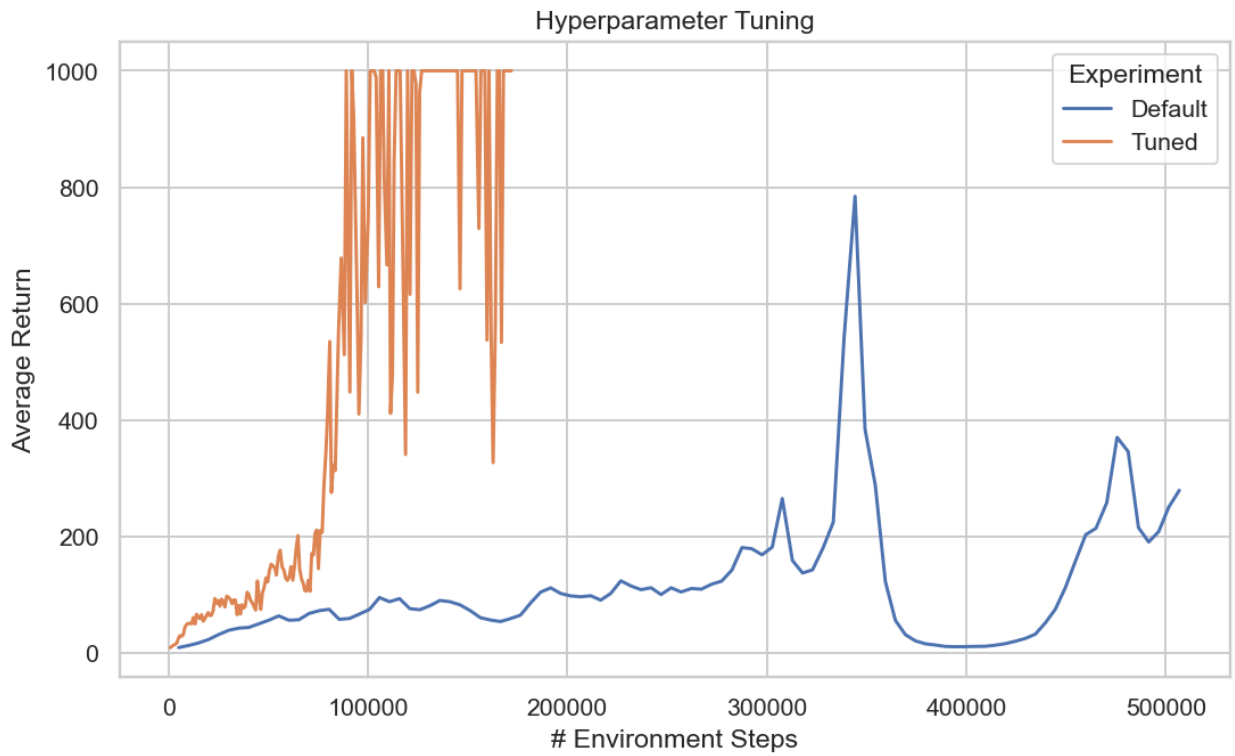
  $\lambda = 0$ corresponds to 1-step TD advantages, with high bias and low variance. $\lambda = 1$ corresponds to Monte Carlo advantage, with low bias and high variance. Given the results of these experiments, bias reduction seems to matter more than variance reduction for learning a good policy.

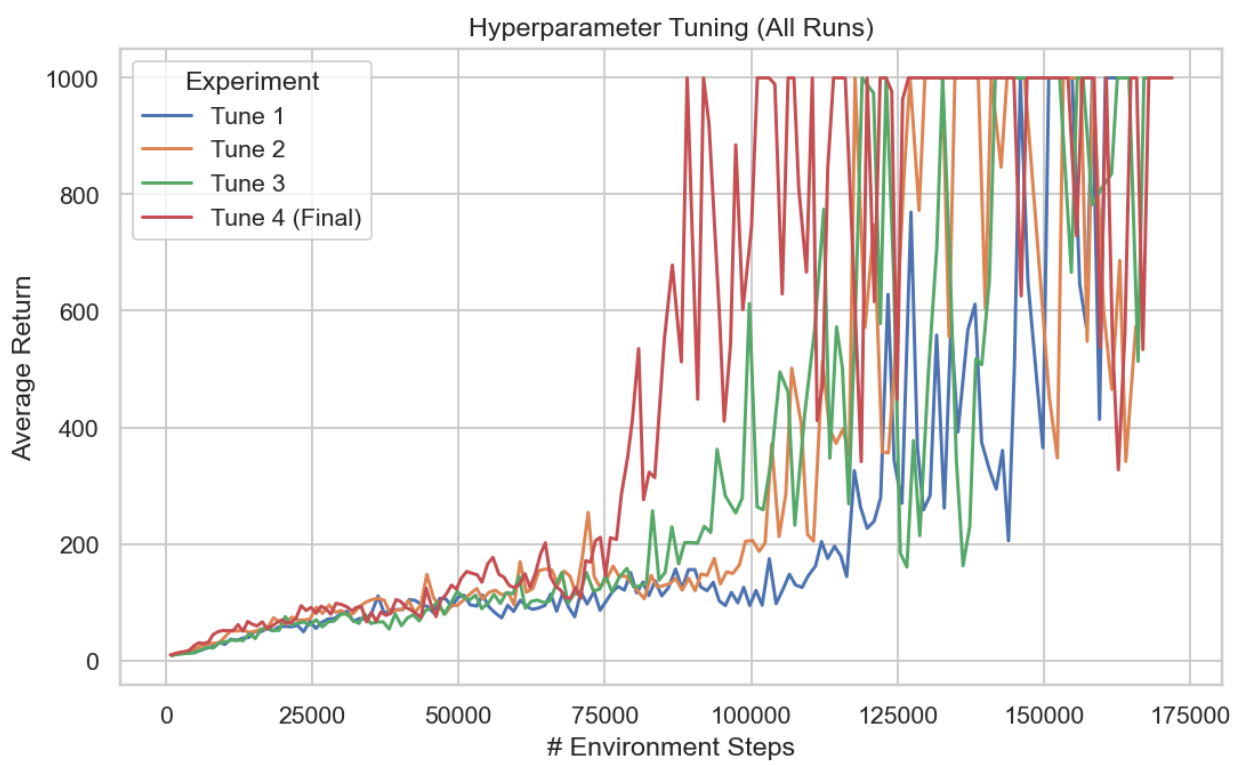
Configuration

```
uv run src/scripts/run.py --env_name LunarLander-v2 --exp_name lunar_lander_lambda0 \
  --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 \
  -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0

uv run src/scripts/run.py --env_name LunarLander-v2 --exp_name lunar_lander_lambda0.95 \
  --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 \
  -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.95

uv run src/scripts/run.py --env_name LunarLander-v2 --exp_name lunar_lander_lambda0.98 \
  --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 \
  -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.98

uv run src/scripts/run.py --env_name LunarLander-v2 --exp_name lunar_lander_lambda0.99 \
  --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 \
```

```
  -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 0.99

uv run src/scripts/run.py --env_name LunarLander-v2 --exp_name lunar_lander_lambda1 \
  --ep_len 1000 --discount 0.99 -n 200 -b 2000 -eb 2000 -l 3 -s 128 \
  -lr 0.001 --use_reward_to_go --use_baseline --gae_lambda 1
```

6 Hyperparameters and Sample Efficiency





Questions

- **Briefly discuss which hyperparameters mattered in your tuning process.**
  - **Batch size**: Had the biggest effect on sample efficiency. Reducing from 5000 to 800 resulted in many more policy updates before 100k steps.
  - **Learning rate**: Default learned a bit too slowly. a moderate increase ($0.002 \rightarrow 0.003$) helped with early progress.
  - **Reward-to-go/normalized advantages**: Helped with learning stability and lower gradient varianve.

  The final run (*Tune 4*) uses a small batch size, RTG, NA, and a slightly higher learning rate to reach 1000 eval reward at $90,877$ environment steps.

Configuration

```
# Default
uv run src/scripts/run.py --env_name InvertedPendulum-v4 --exp_name pendulum \
  -n 100 -b 5000 -eb 1000

uv run src/scripts/run.py --env_name InvertedPendulum-v4 --exp_name pendulum_tune1 \
  -n 150 -b 1000 -eb 1000 -rtg -na --discount 0.99 -lr 0.002

uv run src/scripts/run.py --env_name InvertedPendulum-v4 --exp_name pendulum_tune2 \
  -n 150 -b 1000 -eb 1000 -rtg -na --discount 0.99 -lr 0.003

uv run src/scripts/run.py --env_name InvertedPendulum-v4 --exp_name pendulum_tune3 \
  -n 150 -b 1000 -eb 1000 -rtg -na --use_baseline --gae_lambda 0.95 -lr 0.002 \
  -blr 0.005 -bgs 3 --discount 0.99

# Final Run
uv run src/scripts/run.py --env_name InvertedPendulum-v4 --exp_name pendulum_tune4 \
  -n 180 -b 800 -eb 1000 -rtg -na --discount 0.99 -lr 0.003
```